Invoice and Tax Tracker GUI with Pythonic functional modeling

Inhaltsverzeichnis

1. Project description

Aim

The Invoice and Tax Tracker + Fraud Detector project was conceived as a modular, auditable framework for managing financial documents in environments where compliance, transparency, and fraud prevention are paramount. In today's business landscape, organizations face increasing pressure to ensure that their financial reporting is not only accurate but also defensible under regulatory scrutiny. This project addresses that need by combining robust document parsing, anomaly detection, visualization, and reporting into a single, user-friendly application.

At its core, the system ingests invoices in PDF format and transforms them into structured, machine-readable data. The parsing engine leverages PyMuPDF for text extraction, regex rules for field identification, and OCR fallbacks to capture tampered or hidden values. This hybrid approach ensures resilience against common manipulation techniques, such as altering only the graphical layer of a PDF while leaving the text layer intact. By normalizing amounts and preserving raw text, the parser provides both clean data for downstream analysis and a transparent audit trail for validation.

Once parsed, invoices are passed through the fraud detection module. This component applies a series of plausibility and consistency checks designed to surface anomalies that may indicate fraud or error. Rules include verifying the presence of mandatory fields, checking whether totals are realistic, comparing paid amounts against total prices, detecting suspicious formatting, and validating VAT calculations. The module also compares text-layer values against OCR results to catch discrepancies introduced by tampering. Each anomaly is reported in plain language, making the system's findings explainable and defensible to auditors, executives, and regulators alike.

The graphical user interface, built with PyQt5, orchestrates the workflow and presents results in an accessible manner. Users can import invoices, toggle fraud detection, run batch or single-document analyses, and view anomalies in both textual and visual formats. A fraud heatmap translates anomaly reports into intuitive color-coded grids, allowing users to quickly identify which fields are problematic and how severe the issues are. Debug Mode provides transparency by exposing raw parsed data and anomaly lists, reinforcing the system's commitment to auditability.

Beyond detection, the project emphasizes reporting and compliance. The exporter module enables users to save parsed results to CSV and generate tax summaries that distinguish between valid and flagged invoices. This functionality supports both operational workflows and regulatory reporting requirements. To facilitate demos and training, a test generator creates synthetic invoices with configurable ratios of clean and tampered cases, ensuring that the system can be validated under controlled conditions.

The overarching aim of the project is to demonstrate how advanced analytics and physics-inspired rigor can be embedded into consulting-ready tools that balance technical depth with business clarity. By foregrounding transparency, explainability, and reproducibility, the system provides a defensible framework for fraud detection and tax tracking. It is designed not only to catch anomalies but also to communicate them clearly, bridging the gap between technical analysis and executive decision-making. In doing so, the project showcases how modular AI/ML components can be integrated into client-facing environments to deliver measurable business impact while maintaining compliance and trust

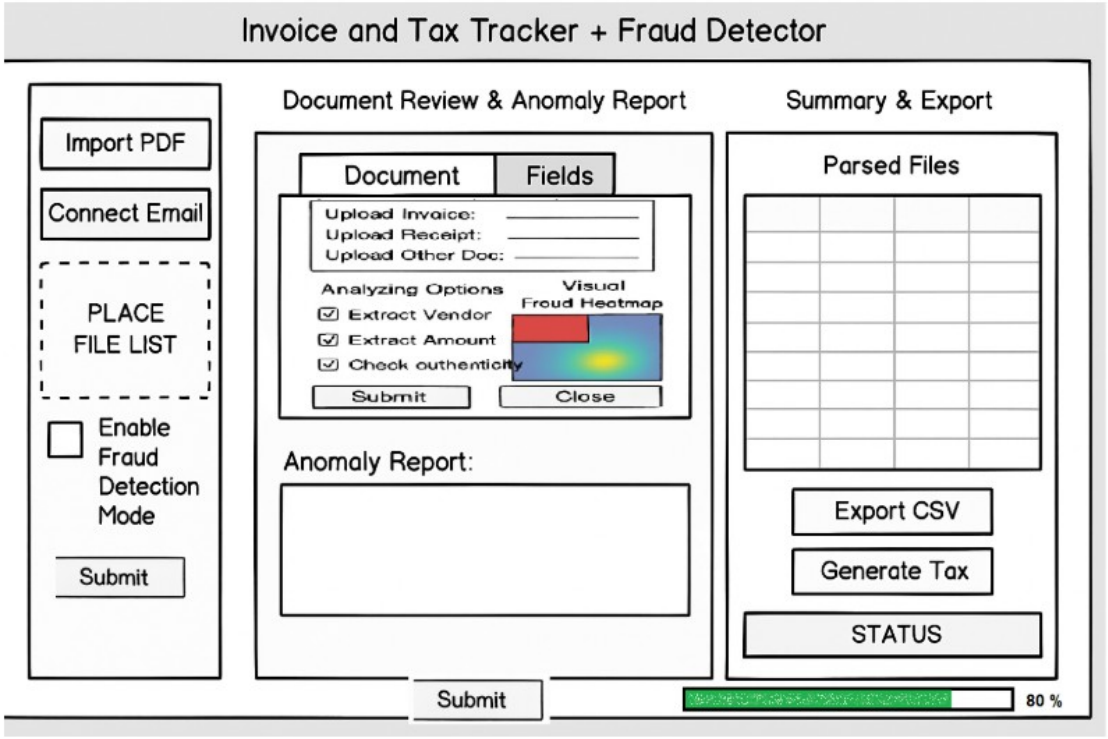
(see References 1 - 3 below).

🎯 Primary Aim

Structure and implement a PyQt5-powered GUI for detecting fraudulent content in invoices and tax forms with built-in logic to route the problem to regex and OCR parsers prior to subjecting such parsed output to fraud detection modules, both from a UX and reporting (compliance) architecture perspective.

2. GUI and algorithmic concepts

In the following we address our full GUI sketch, a clean, structured layout for our PyQt5-based Tax/Invoice Tracker and Fraud detection GUI. It includes all the key modules we discussed: objective and constraint input, method selection (OCR parsing, fraud detection mode), result display, visualization, and diagnostics.



GUI Architecture Overview

The interface is divided into three main panels:

1. **Left Panel** – Document Import & Fraud Toggle
2. **Center Panel** – Document Review & Anomaly Report
3. **Right Panel** – Summary Table, Export Options & Heatmap

1. Left Panel: Document Import & Fraud Toggle

File Import

- **Import PDF**: Opens a file dialog to select one or more invoice/receipt PDFs.
- **Connect Email**: Initiates email integration (e.g., IMAP or Outlook API) to fetch receipts from inbox.

📋 File List Display

- **PLACE FILE LIST**: A scrollable area showing imported files with status indicators (e.g., parsed, flagged, error).

🛡️ Fraud Detection Toggle

- **Enable Fraud Detection Mode**: Checkbox that activates anomaly checks and visual heatmap generation.

🟢 Submit Button

- Begins parsing and analysis of all imported documents using selected options.

🔶 2. Center Panel: Document Review & Anomaly Report

📁 Tabs

- **Document Tab**: Displays raw document preview (PDF snapshot or OCR text).
- **Fields Tab**: Shows extracted fields like vendor, amount, date, category.

📤 Upload Fields

- **Upload Invoice / Receipt / Other Doc**: Optional manual upload fields for specific document types.

🧠 Analyzing Options

- **Extract Vendor**: Enables vendor name extraction via NLP or regex.
- **Extract Amount**: Parses total, subtotal, and tax values.
- **Extract Category**: Classifies expense type (e.g., travel, meals, office supplies).

🔍 Fraud Detect Settings

- **Check Authenticity**: Runs heuristic and visual checks for tampering, mismatched totals, or duplicate entries.

🔥 Visual Fraud Heatmap

- Displays a color-coded overlay indicating suspicious regions in the document:
  - **Red**: High anomaly score (e.g., altered totals)
  - **Yellow**: Moderate concern (e.g., missing fields)
  - **Blue/Green**: Low risk or clean areas

📄 Anomaly Report

- Textual summary of detected issues:
  - "Invoice #123 has mismatched totals"
  - "Vendor name not recognized"
  - "Font inconsistency detected in amount field"

🟢 Submit / ❌ Close Buttons

- **Submit**: Re-analyzes current document with updated settings.
- **Close**: Clears current document or exits the review panel.

🔷 3. Right Panel: Summary & Export

📊 Parsed Files Table

- Displays all processed documents with columns:
  - File Name
  - Vendor
  - Amount
  - Status (e.g., Clean, Flagged, Error)

📤 Export Options

- **Export CSV**: Saves parsed data for accounting or audit purposes.
- **Generate Tax**: Creates a pre-filled tax form or summary report (PDF or structured format).

🟦 STATUS Indicator (Bottom Right)

🔍 Purpose:

- Displays **real-time feedback** about the system's current state or recent actions.

🧠 Typical Messages:

- ✅ "Parsing complete: 5 documents processed"
- ⚠️ "2 invoices flagged for fraud review"
- ❌ "Error: Receipt_04.pdf could not be parsed"
- 📤 "Export successful: invoices_2025.csv saved"

🔄 Behavior:

- Updates dynamically after each major action:
  - File import
  - Fraud detection
  - Export
  - Tax report generation

3. GUI design and its interaction flow

3.1 `gui.py`:

🎛️ High-Level Purpose

`gui.py` defines the **InvoiceTrackerApp**, a PyQt5 desktop application that:

- Imports and lists PDF invoices.
- Parses them (`parser.py`).
- Detects anomalies (`fraud.py`).
- Displays parsed fields and anomaly reports.
- Provides debug output and a fraud heatmap.
- Exports results to CSV and generates tax reports.
- Simulates test invoices for demos.

It's essentially the **front-end controller** that ties together our parsing, fraud detection, visualization, and export modules.

3.2 `parser.py`:

🎛️ High-Level Purpose

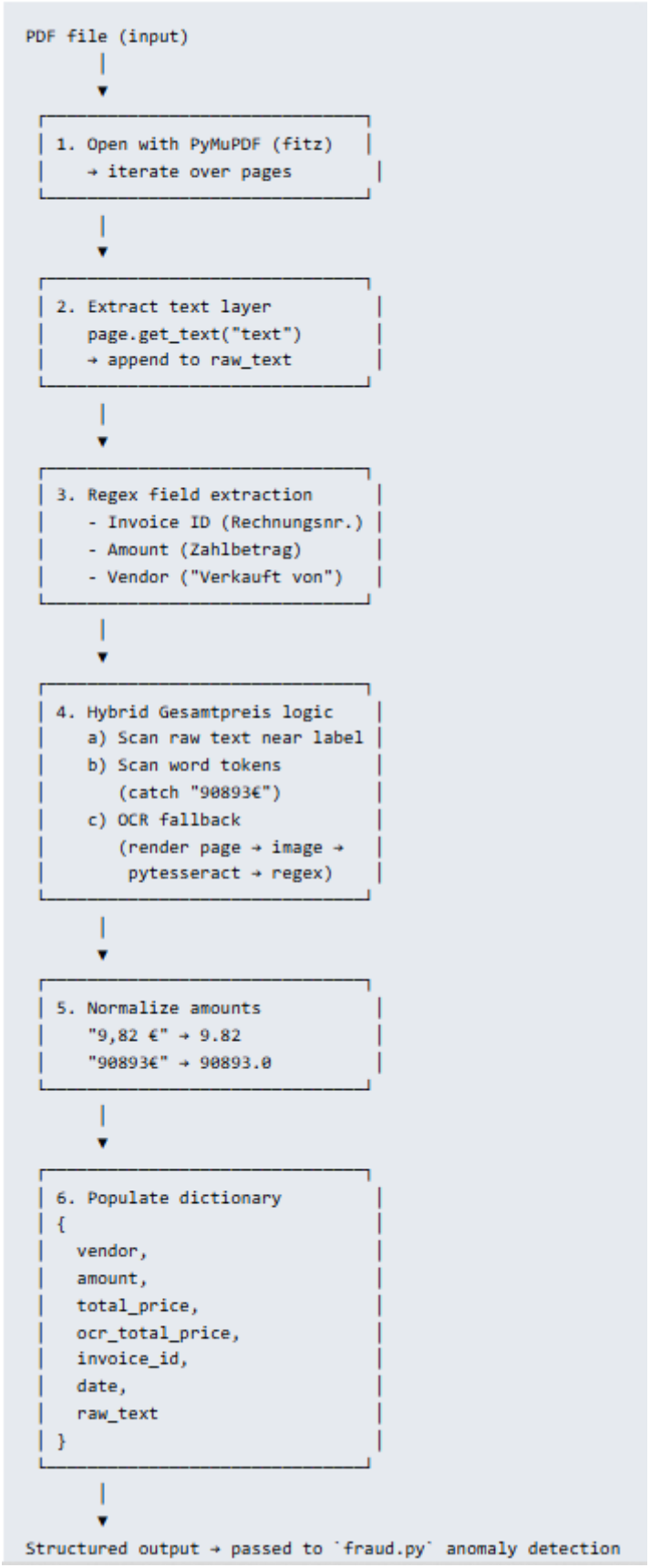
`parser.py` ingests a PDF invoice, reads its text layer (and optionally OCRs the rendered page), and returns a dictionary with key fields like vendor, invoice ID, amount, total price, and raw text. It's designed to be robust against tampering by combining text parsing, regex, and OCR fallback.

🎯 Executive Takeaway

- **Parser is the foundation:** It transforms messy invoice PDFs into structured fields.
- **Hybrid extraction:** Combines text layer, word tokens, and OCR to catch tampering.
- **Auditability:** Keeps `raw_text` so anomalies can be traced back to source.
- **Flexibility:** Works for both clean and manipulated invoices.

Here's a clear **textual flow diagram** of how our `parser.py` processes an invoice PDF:

`parser.py` Flow Diagram (Textual)



🎯 Executive Narrative

- **Step 1–2:** We open the PDF and capture its text layer.
- **Step 3:** Regex rules extract invoice ID, vendor, and Zahlbetrag.
- **Step 4:** Gesamtpreis is extracted with a hybrid strategy: text layer, word tokens, and OCR fallback.
- **Step 5:** All amounts are normalized into floats for comparison.
- **Step 6:** Results are packaged into a dictionary for downstream fraud detection.

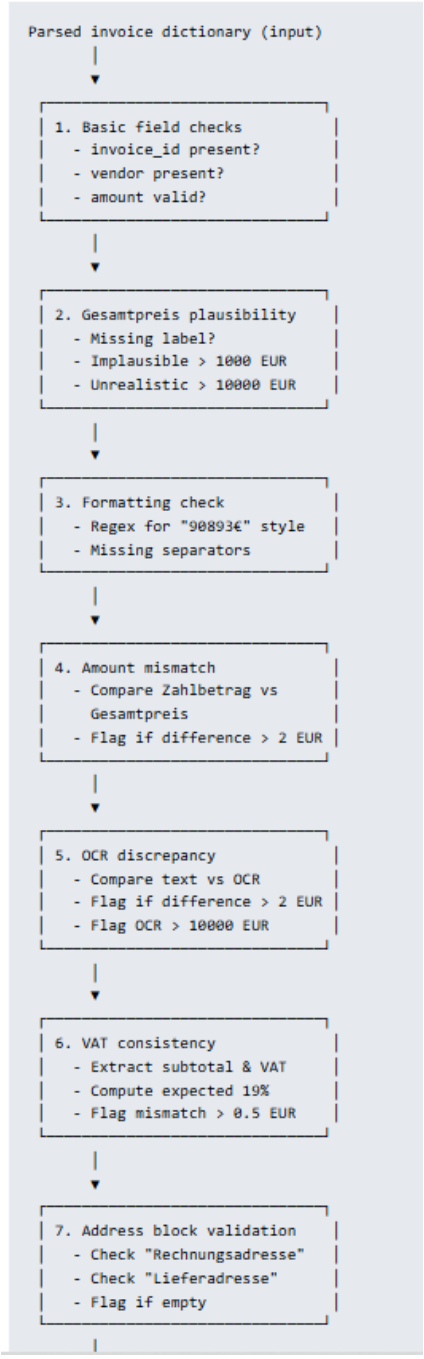
👉 This flow shows how the parser is **modular, auditable, and tamper-resistant** — exactly the qualities executives care about.

3.3 `fraud.py`:

🎛️ High-Level Purpose

`fraud.py` takes the structured dictionary from `parser.py` and applies a series of anomaly detection rules. It checks for missing fields, implausible totals, mismatches between amounts, OCR discrepancies, VAT consistency, and empty address blocks. The output is a list of anomaly messages that can be displayed in our GUI.

📄 `fraud.py` Flow Diagram (Textual)



```
        ▼
List of anomaly messages (output)
```

🎯 Executive Narrative

- **Step 1–2:** We validate basic fields and check if totals are plausible.
- **Step 3–4:** We catch suspicious formatting and mismatches between Zahlbetrag and Gesamtpreis.
- **Step 5:** We compare text layer vs OCR to detect hidden manipulations.
- **Step 6:** We verify VAT consistency against expected 19%.
- **Step 7:** We ensure addresses are present after their labels.

👉 This flow shows how `fraud.py` enforces **auditability, plausibility, and consistency checks** — making fraud detection explainable and defensible.

Here's the **end-to-end pipeline view** that combines `parser.py` and `fraud.py` into one coherent story.

📄 End-to-End Invoice Analysis Pipeline

```
PDF Invoice (input)
        |
        ▼
 ┌──────────────────────────────┐
 | Parser (parser.py)           |
 |                              |
 | 1. Open PDF with PyMuPDF     |
 | 2. Extract text layer        |
 | 3. Regex field extraction    |
 |    - Invoice ID              |
 |    - Vendor                  |
 |    - Zahlbetrag (amount)     |
 | 4. Hybrid Gesamtpreis logic  |
 |    a) Text near "Gesamtpreis"|
 |    b) Word tokens (catch 90893€)|
 |    c) OCR fallback           |
 | 5. Normalize amounts         |
 | 6. Build structured dictionary|
 └──────────────────────────────┘
        |
        ▼
 ┌──────────────────────────────┐
 | Fraud Detector (fraud.py)    |
 |                              |
 | 1. Basic field checks        |
 |    - Invoice ID, vendor, amount|
 | 2. Gesamtpreis plausibility  |
 |    - Missing, implausible,   |
 |      unrealistic thresholds  |
 | 3. Formatting check          |
 |    - Suspicious "90893€" style|
 | 4. Amount mismatch           |
 |    - Zahlbetrag vs Gesamtpreis|
 | 5. OCR discrepancy           |
 |    - Text vs OCR comparison  |
 | 6. VAT consistency           |
 |    - Expected 19% vs reported|
 | 7. Address block validation  |
 |    - Billing & shipping present|
 └──────────────────────────────┘
        |
        ▼
Anomaly Report (output)
- List of flagged issues
- Debug Mode shows raw parsed data
- Heatmap visualizes anomalies
```

🎯 Executive Narrative

- **Step 1:** The parser transforms messy PDFs into structured fields using text, regex, and OCR.
- **Step 2:** The fraud detector applies plausibility, consistency, and discrepancy rules.
- **Step 3:** Results are surfaced in the GUI as anomaly reports, debug output, and heatmaps.
- **Business Impact:** This pipeline ensures invoices are **transparent, auditable, and fraud-resistant**, supporting compliance and reducing risk.

3.4 📄 `exporter.py`

🎲 Purpose

This module enables users to **export parsed invoice data** and **generate tax summaries** directly from the GUI's table widget. It bridges the analysis results with external reporting formats (CSV).

👉 *Executive narrative:* "This function produces a tax summary that distinguishes clean invoices from flagged ones, supporting compliance and audit readiness."

3.5 📄 `heatmap.py`

🎲 Purpose

This module provides a **visual fraud heatmap**. It maps anomalies to a grid and displays them with intensity colors, making fraud detection more intuitive.

👉 *Executive narrative:* "The heatmap translates anomaly text into a visual grid, so users can instantly see which fields are suspicious and how severe the anomalies are."

🎯 Combined Takeaway

- `exporter.py`: Converts analysis results into CSVs for compliance and tax reporting.
- `heatmap.py`: Provides a visual fraud map, making anomalies intuitive and executive-friendly.

Together, they ensure our system isn't just detecting fraud — it's **communicating results clearly** in both structured reports and visual dashboards.

3.6 📄 `test_generator.py`

🎲 Purpose

This module programmatically generates **synthetic invoice PDFs** for testing and demos. It allows you to simulate clean vs tampered invoices with randomized vendors, categories, and amounts.

🎯 Executive Narrative

"This module lets us generate synthetic invoices on demand, mixing clean and tampered cases. It's perfect for demos, training, and stress-testing the fraud detection pipeline."

3.7 📄 `main.py`

🎲 Purpose

This is the **entry point** of your application. It launches the PyQt5 GUI defined in `gui.py`.

🧩 Workflow

- Imports `QApplication` from PyQt5 and your `InvoiceTrackerApp`.
- Creates the application object (`app = QApplication(sys.argv)`).
- Instantiates the main window (`window = InvoiceTrackerApp()`).
- Shows the window (`window.show()`).
- Starts the Qt event loop (`sys.exit(app.exec_())`).

🎯 Executive Narrative

"This file is the launcher. It initializes the Qt application and opens the Invoice Tracker GUI. Everything else — parsing, fraud detection, exports — is orchestrated from here."

3.8 📄 `requirements.txt`

🎲 Purpose

Defines the Python dependencies needed to run your system. This ensures reproducibility and easy setup.

🧩 Dependencies

- **PyQt5** → GUI framework.
- **fpdf** → Generate synthetic test invoices.
- **PyMuPDF** → Parse PDF text and structure.
- **matplotlib** → Render fraud heatmaps.
- **numpy** → Grid math for heatmaps.
- **pytesseract** → OCR fallback for tampered invoices.
- **Pillow** → Image handling for OCR.

🎯 Executive Narrative

"This file lists all required packages. It guarantees that anyone can install dependencies and run the system consistently."

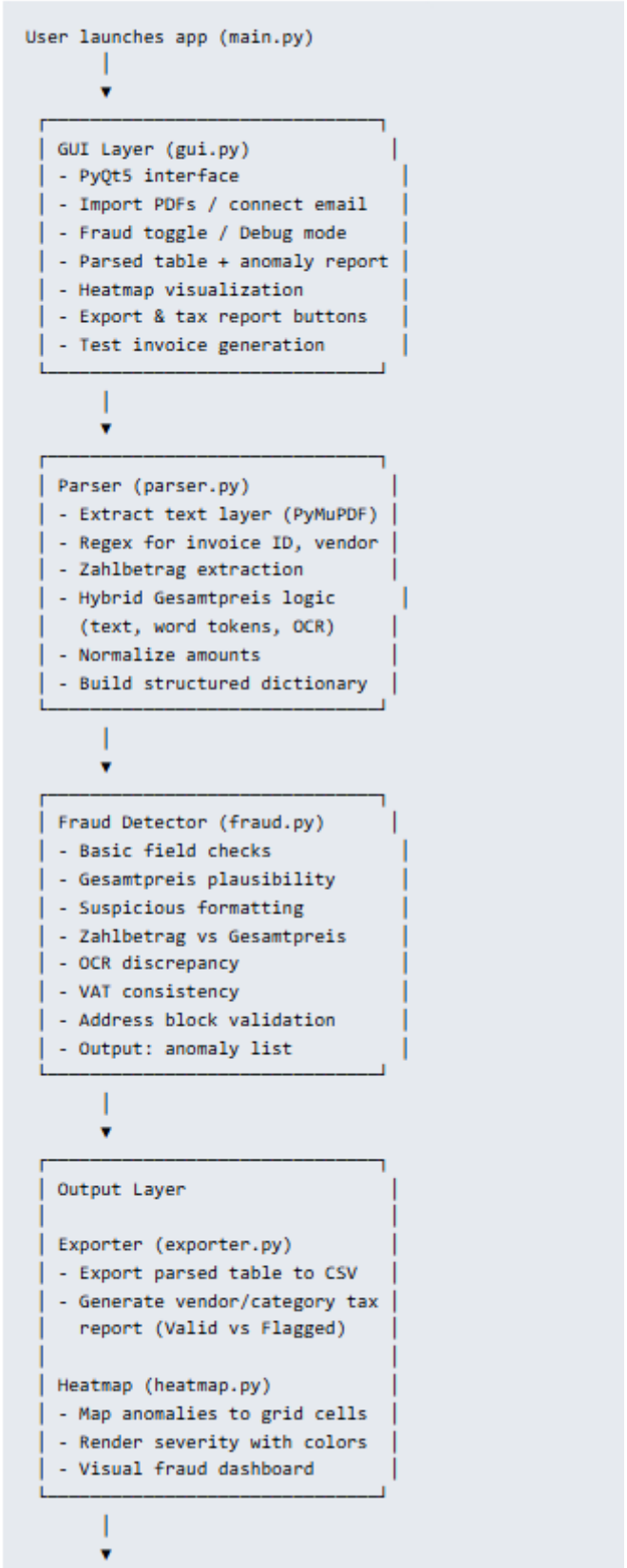
🔗 Combined Role

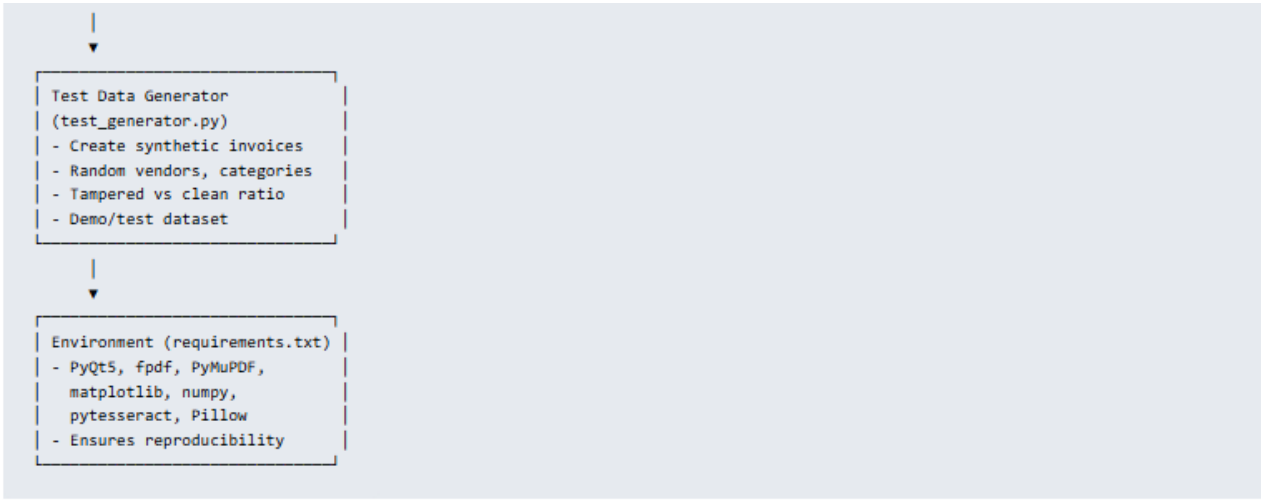
- `test_generator.py` → Creates demo data.
- `main.py` → Launches the GUI.
- `requirements.txt` → Ensures environment reproducibility.

Together, they make our system **demo-ready, portable, and reproducible**.

In the following we conclude with the **full architectural overview** of our system — a textual diagram that ties together all the modules (`parser.py`, `fraud.py`, `gui.py`, `exporter.py`, `heatmap.py`, `test_generator.py`, `main.py`, plus `requirements.txt`).

3.9 🏗️ End-to-End System Architecture





🎯 Executive Narrative

- **GUI Layer:** The user interacts through a PyQt5 interface — importing invoices, toggling fraud detection, viewing anomalies, and exporting results.
- **Parser:** Converts messy PDFs into structured fields using text extraction, regex, and OCR fallback.
- **Fraud Detector:** Applies plausibility, consistency, and discrepancy rules to flag anomalies.
- **Output Layer:** Results are communicated clearly — structured CSVs for compliance, tax summaries for reporting, and heatmaps for intuitive visualization.
- **Test Generator:** Provides synthetic clean/tampered invoices for demos and validation.
- **Environment:** Requirements ensure reproducibility across machines.
- **Main Launcher:** Starts the application and ties everything together.

This architecture shows a **modular, auditable, and demo-ready system**: from ingestion to fraud detection to reporting and visualization.

4. Heatmap coloring scheme

The heatmap is a **visual diagnostic tool** that highlights potential anomalies in invoice data. Here's how it works:

🎨 Color Mapping

- **Red / Bright Yellow**: Indicates regions with **high anomaly intensity** — these are areas where fraud signals (like tampering, missing fields, or suspicious keywords) are concentrated.
- **Orange / Mid-Tones**: Moderate anomaly likelihood — possibly inconsistent data or borderline suspicious entries.
- **Dark Shades (Black / Deep Red)**: Low or no anomaly presence — these regions are considered clean or normal.

🧠 Data Simulation

The heatmap uses a synthetic 10×10 matrix (`np.random.rand(10, 10)`) to simulate anomaly distribution. When anomalies are detected:

```
data[0:3, 0:3] += 2  # artificially intensifies top-left region
```

This creates a **clustered hotspot** to visually represent fraud detection activity.

🧪 What It Represents

- The heatmap doesn't map to specific fields (like vendor or amount) — it's a **symbolic visualization** of anomaly density.
- It's meant to give users a **quick visual cue**: if the heatmap lights up, the document likely contains fraud indicators.

Also, understanding the distinction between **invoice**, **receipt**, and **other document** is key to organizing and evaluating financial records accurately. Here's how they differ in purpose and structure:

📜 Invoice vs. Receipt vs. Other Document

Type	Purpose	Issued By	Key Features
Invoice	Request for payment	Seller or service provider	Includes due date, itemized charges, tax, total amount
Receipt	Proof of payment already made	Seller or payment processor	Shows payment confirmation, method, date, and amount
Other Document	Supplementary or unrelated financial info	Varies (e.g., bank, employer)	Could be contracts, delivery notes, tax forms, etc.

🔍 Invoice

- Sent **before** payment
- Used for billing and accounting
- May include payment terms (e.g., Net 30)
- Often triggers fraud detection (e.g., inflated charges)

✅ Receipt

- Issued **after** payment is received
- Confirms transaction completion
- Less likely to be fraudulent, but still worth verifying (e.g., duplicate charges)

📂 Other Document

- Catch-all for anything that's not a formal invoice or receipt
- Examples: delivery slips, tax letters, bank statements, warranty forms
- May contain useful metadata or context for fraud analysis

🧠 In Our GUI

When we upload:

- **Invoice** → triggers full parsing + fraud evaluation
- **Receipt** → confirms payment and cross-checks invoice
- **Other Doc** → stored for reference, may be parsed if relevant

5. Modularized Pythonic implementation (Tax-Invoice-GUI)

This is a comprehensive, notebook-ready Markdown explanation of major optimization, parsing and OCR techniques our GUI should support. Each section includes mathematical foundations, algorithmic steps, and illustrative examples.

Project Structure

```
invoice_tracker_gui/
├── main.py
├── gui.py
├── parser.py
├── fraud.py
├── heatmap.py
├── exporter.py
├── test_generator.py
└── requirements.txt
```

main.py

```python
from PyQt5.QtWidgets import QApplication
from gui import InvoiceTrackerApp
import sys

if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = InvoiceTrackerApp()
    window.show()
    sys.exit(app.exec_())
```

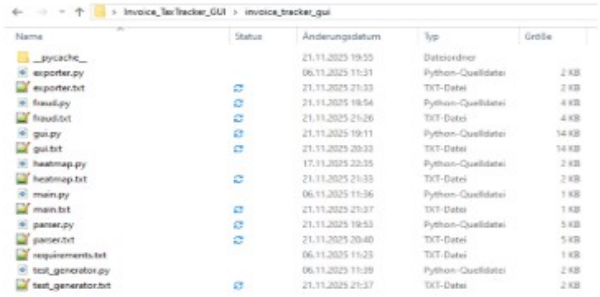
The full Pythonic code can be accessed via https://github.com/NenadBalaneskovic/ExternalProjects/blob/main/Invoice_TaxTracker_GUI/Invoice_TaxTracker_GUI.md (sections 2.1 – 2.6 therein).

6. Results and conclusions

6.1 📊 Start the Tax-Invoice-GUI

✅ Step 1: Download the folder

Download the main folder 📁 Invoice_TaxTracker_GUI which has the following structure:



✅ Step 2: Run the jupyter runner

Run the jupyter file "**run_gui.ipynb**" (subfolder "**invoice_tracker_gui**") in VS Code or Jupyter notebook.

✅ Step 3: Interact with the Tax-Invoice-GUI

Interact with the Tax-Invoice-GUI by providing reasonable inputs in the left half-plane and pressing the "**Submit**"-button.

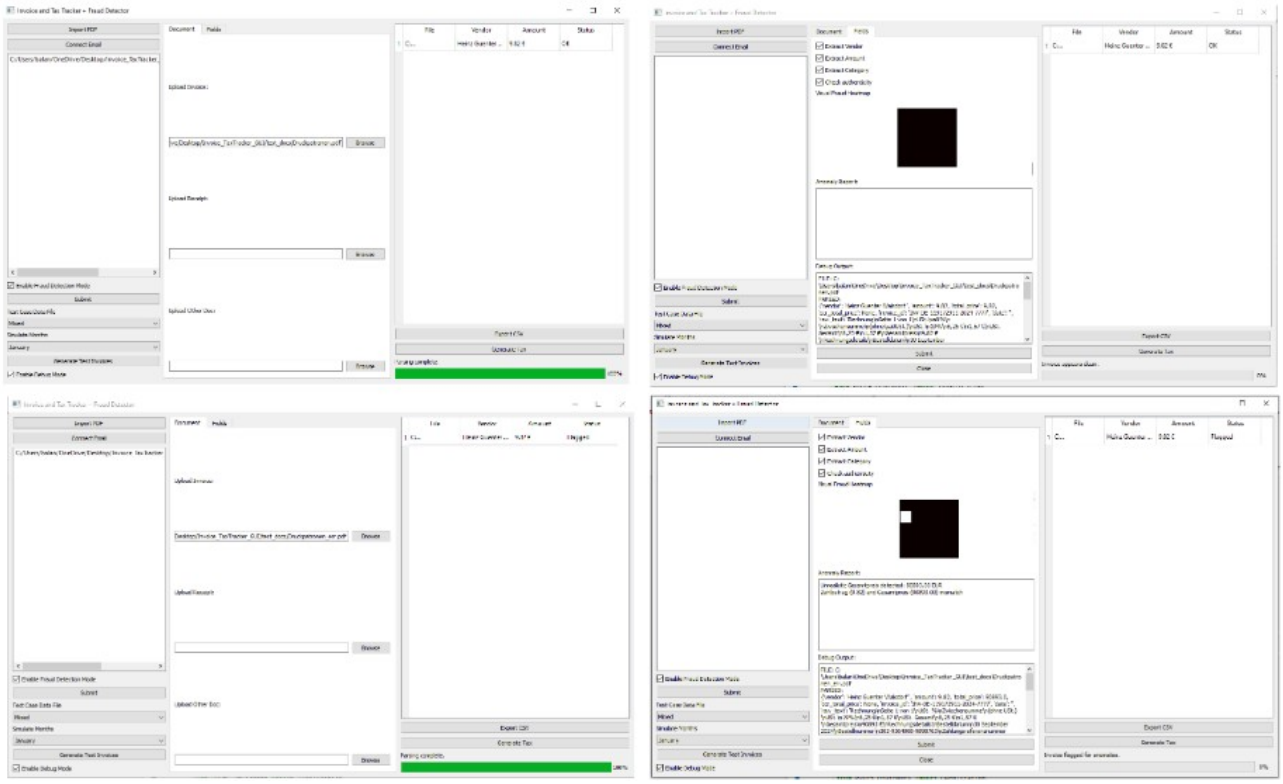
6.2 🧠 Interpretation of results

🧠 This Tax-Invoice-GUI...

- Accepts user inputs:
  - invoices and tax forms as PDFs
  - extraction field specifications
- Displays fraud detection results emerging from the successive application of different parsing and OCR methods and renders them as 2D-plots (heatmaps) within the GUI-plane
- Stores obtained results and characterizations as csv files (see figure below)

```
1  File,Vendor,Amount,Status
2  C:/Users/balan/OneDrive/Desktop/Invoice_TaxTracker_GUI/test_docs/Druckpatronen_err.pdf,Heinz Guenter Walsdorf,9.82,Flagged
3
```

- Automatically displays the fraud detection heatmap associated with selected (extracted) invoice fields (see figure below).



6.3 🏁 Final Thoughts

The Invoice and Tax Tracker + Fraud Detector project demonstrates how modular, auditable design can transform the way organizations handle financial documentation. By integrating parsing, anomaly detection, visualization, and reporting into a single cohesive framework, the system provides a transparent and defensible approach to invoice management. Each component — from the hybrid parser that combines text extraction and OCR, to the fraud detector that applies plausibility and consistency checks, to the GUI that surfaces results in both textual and visual formats — contributes to a pipeline that is robust, explainable, and business-ready.

The project's strength lies in its balance of technical rigor and practical usability. The parser ensures that even tampered invoices are captured accurately, while the fraud module translates complex validation rules into clear anomaly reports. The GUI empowers users to interact with the system intuitively, toggling fraud detection, running batch or single analyses, and exporting results for compliance. The heatmap visualization adds an executive-friendly layer of insight, making anomalies immediately visible and understandable. Meanwhile, the exporter and tax report generator bridge the gap between detection and compliance, ensuring that flagged anomalies can be incorporated into structured reporting workflows. The test generator further enhances the system's utility by providing synthetic datasets for demos, training, and validation.

Collectively, these modules embody the project's overarching aim: to deliver a consulting-ready solution that foregrounds transparency, auditability, and reproducibility. In environments where regulatory scrutiny and fraud risk are ever-present, this system offers a defensible framework that not only detects anomalies but also explains them in a way that builds trust with auditors, executives, and clients. By embedding explainability and validation into every stage, the project showcases how advanced analytics can be deployed responsibly and effectively.

Beyond its immediate functionality, the project also serves as a proof of concept for how physics-inspired rigor and AI/ML best practices can be integrated into client-facing tools. It highlights the importance of modular design, where each component can be refined independently yet contributes to a coherent whole. It demonstrates how transparency features like Debug Mode and anomaly heatmaps can elevate technical outputs into governance-ready artifacts. Most importantly, it illustrates how technical depth can be translated into business clarity, ensuring that complex detection logic is narratable and actionable at the executive level.

In conclusion, the Invoice and Tax Tracker + Fraud Detector is more than a technical prototype; it is a strategic framework for bridging compliance, fraud prevention, and executive communication. It underscores the value of explainable AI in consulting contexts, where trust and defensibility are as critical as detection accuracy. By combining robust parsing, anomaly detection, visualization, and reporting, the project delivers a comprehensive solution that is not only technically sound but also aligned with the broader goals of governance, transparency, and business impact.

7. User Manual

Here is a comprehensive, step-by-step **User Manual** for our **Invoice and Tax Tracker + Fraud Detector GUI**. It covers every feature, input/output behavior, and practical examples to help users navigate and test the system confidently.

🧠 Pre-Step 1: Install Tesseract OCR Engine

🔧 Using the official installer

1. Go to the official Tesseract GitHub page: https://github.com/tesseract-ocr/tesseract
2. Scroll down to the **Windows** section and download the installer from UB Mannheim builds.
3. Download the `.exe` installer (e.g., `tesseract-ocr-w64-setup-v5.3.3.20231005.exe`).
4. Run the installer and follow the setup instructions.
5. During installation, **note the installation path** (e.g., `C:\Program Files\Tesseract-OCR`).

🧠 Pre-Step 2: Add Tesseract to System PATH

1. Open **Start Menu** → search for **Environment Variables** → click **Edit the system environment variables**.
2. In the **System Properties** window, click **Environment Variables**.
3. Under **System variables**, find and select `Path`, then click **Edit**.
4. Click **New**, and add the path to the Tesseract installation folder (e.g., `C:\Program Files\Tesseract-OCR`).
5. Click **OK** to save and close all dialogs.

🧠 Pre-Step 3: Install pytesseract via pip

Open your command prompt and run:

```
pip install pytesseract
```

🧠 Pre-Step 4: Verify Installation

Create a quick Python script to test:

```
import pytesseract
from PIL import Image

# Optional: specify path if not in system PATH
pytesseract.pytesseract.tesseract_cmd = r'C:\Program Files\Tesseract-OCR\tesseract.exe'

img = Image.open('sample_image.png')  # Replace with your image path
text = pytesseract.image_to_string(img)
print(text)
```

✅ You're all set!

You can now use Tesseract OCR in your Python projects.

📌 Overview

This application allows users to:

- Import and analyze invoices, receipts, and other financial documents
- Detect potential fraud using rule-based heuristics
- Visualize anomalies via a heatmap
- Generate test data for simulation
- Export results for tax and audit purposes

🖥️ GUI Layout

Section	Description
Left Panel	File import, email connection, fraud toggle, test data generation
Center Panel	Document upload tabs and fraud analysis options
Right Panel	Parsed results table, export buttons, status and progress bar

🧭 Step-by-Step Guide

1. 📂 Importing Documents

Option A: Manual Import

- Click **"Import PDF"**
- Select one or more `.pdf` files
- Files will appear in the **file list**

Option B: Upload via Tabs

- Go to **"Document"** tab
- Use **"Browse"** buttons next to:
  - Upload Invoice
  - Upload Receipt
  - Upload Other Doc

> ✅ These fields are used for single-document analysis in the **"Fields"** tab.

2. 📧 Email Integration (Placeholder)

- Click **"Connect Email"**
- Currently displays: *"Email integration not yet implemented."*

3. 🧪 Generating Test Invoices

- Select a **test type**:
  - Mixed
  - Clean Only
  - Tampered Only
- Choose a **month** from the dropdown
- Click **"Generate Test Invoices"**
- 10 synthetic PDFs will be created and added to the file list

> Example: `test_invoice_January_3_tampered.pdf`

4. 🕵️ Enabling Fraud Detection

- Toggle **"Enable Fraud Detection Mode"**
- When enabled, the system will:
  - Check for missing fields
  - Detect suspicious keywords (e.g., "manipuliert")
  - Flag font mismatches or altered vendor names

5. 🚀 Running the Analysis

- Click **"Submit"** (left panel)
- Each file is parsed and evaluated
- Results appear in the **Parsed Table**:
  - File path
  - Vendor name
  - Amount
  - Status: `OK` or `Flagged`

> Progress bar shows completion percentage
> Status label updates to "Parsing complete."

6. 🔍 Analyzing a Single Document

- Go to **"Fields"** tab
- Select a file via the upload fields
- Check desired options:
  - Extract Vendor
  - Extract Amount
  - Extract Category
  - Check Authenticity
- Click **"Submit"**
- Output:
  - **Anomaly Report** (textual)
  - **Visual Heatmap** (fraud intensity)

7. 📤 Exporting Results

Export CSV

- Click **"Export CSV"**
- Saves the parsed table to a `.csv` file

Generate Tax

- Click **"Generate Tax"**
- Creates a summary of:
  - Total amount per vendor
  - Flagged vs. valid totals
- Saves to a `.csv` file

📈 Heatmap Interpretation

- **Bright Red/Yellow**: High anomaly density
- **Orange**: Moderate suspicion
- **Dark**: Clean zones
- Symbolic visualization — not tied to document coordinates

📂 Input Examples

✅ Clean Invoice

```
Rechnungsnummer: INV-DE-2025-0001
Verkauft von: Heinz Guenter Walsdorf
Zahlbetrag: 9,82 EUR
```

⚠️ Tampered Invoice

```
Verkauft von: Heinz Günter W@lsdorf
Hinweis: Dieses Dokument wurde manipuliert.
```

📤 Output Examples

File	Vendor	Amount	Status
test_invoice_January_1_clean.pdf	Heinz Guenter Walsdorf	9.82	OK
test_invoice_January_3_tampered.pdf	Heinz Günter W@lsdorf	9.82	Flagged

🧠 Tips & Notes

- Use "**Generate Test Invoices**" to validate fraud detection logic
- Use "**Fields**" **tab** for deep dive into a single document
- Replace `€` with `EUR` in test PDFs to avoid encoding issues
- OCR fallback is triggered if text extraction fails

Finally, we offer a crisp **demo flow** of our GUI. It highlights the value chain from ingestion to anomaly detection and reporting:

8. References

1. (Py-)tesseract package: https://github.com/tesseract-ocr/tesseract, https://pypi.org/project/pytesseract/, https://builtin.com/data-science/python-ocr, https://www.analyticsvidhya.com/blog/2024/04/ocr-libraries-in-python/ and UB Mannheim builds.

2. A. Meister , T. Sonar: "Numerik", 1st Ed. Springer-Spektrum (2019); S. Chapra, R. Canale: "Numerical Methods for Engineers", Mcgraw-Hill, 6th Edition (2010).

3. J. Kilty, A. M. McAllister: "Mathematical Modeling and Applied Calculus", 1st Ed. Oxford University Press (2018).

4. U. Kockelkorn: "Statistik für Anwender", 1st Ed. Springer (2012), s. chapters 7 - 8.

5. Robert H. Shumway, David S. Stoffer: "Time Series Analysis and Its Applications with R Examples", Springer (2011).

6. Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, Jonathan Taylor: "An Introduction to Statistical Learning with Applications in Python", Springer (2023).

7. Cornelis W. Oosterlee, Lech A. Grzelak: "Mathematical Modeling and Computation in Finance with Exercises and Python and MATLAB Computer Codes", World Scientific (2020).

8. Richard Szeliski: "Computer Vision - Algorithms and Applications", Springer (2022).

9. Anthony Scopatz, Kathryn D. Huff: "Effective Computation in Physics - Field Guide to Research with Python", O'Reilly Media (2015).

10. Alex Gezerlis: "Numerical Methods in Physics with Python", Cambridge University Press (2020).

11. Gary Hutson, Matt Jackson: "Graph Data Modeling in Python. A practical guide", Packt-Publishing (2023).

12. Hagen Kleinert: "Path Integrals in Quantum Mechanics, Statistics, Polymer Physics, and Financial Markets", 5th Edition, World Scientific Publishing Company (2009).

13. Peter Richmond, Jurgen Mimkes, Stefan Hutzler: "Econophysics and Physical Economics", Oxford University Press (2013).

14. A. Coryn , L. Bailer Jones: "Practical Bayesian Inference A Primer for Physical Scientists", Cambridge University Press (2017).

15. Avram Sidi: "Practical Extrapolation Methods - Theory and Applications", Cambridge university Press (2003).

16. Volker Ziemann: "Physics and Finance", Springer (2021).

17. Zhi-Hua Zhou: "Ensemble methods, foundations and algorithms", CRC Press (2012).

18. B. S. Everitt, et al.: "Cluster analysis", Wiley (2011).

19. Lior Rokach, Oded Maimon: "Data Mining With Decision Trees - Theory and Applications", World Scientific (2015).

20. Bernhard Schölkopf, Alexander J. Smola: "Learning with kernels - support vector machines, regularization, optimization and beyond", MIT Press (2009).

21. Johan A. K. Suykens: "Regularization, Optimization, Kernels, and Support Vector Machines", CRC Press (2014).

22. Sarah Depaoli: "Bayesian Structural Equation Modeling", Guilford Press (2021).

23. Rex B. Kline: "Principles and Practice of Structural Equation Modeling", Guilford Press (2023).

24. Ekaterina Kochmar: "Getting Started with Natural Language Processing", Manning (2022).

25. Rex B. Kline: "Principles and Practice of Structural Equation Modeling", Guilford Press (2023).

26. Ekaterina Kochmar: "Getting Started with Natural Language Processing", Manning (2022).

27. Jakub Langr, Vladimir Bok: "GANs in Action", Computer Vision Lead at Founders Factory (2019).

28. David Foster: "Generative Deep Learning", O'Reilly(2023).

29. Rowel Atienza: "Advanced Deep Learning with Keras: Applying GANs and other new deep learning algorithms to the real world", Packt Publishing (2018).

30. Josh Kalin: "Generative Adversarial Networks Cookbook", Packt Publishing (2018).

31. Thomas Haslwanter: „Hands-on Signal Analysis with Python: An Introduction“, Springer (2021).

32. Jose Unpingco: „Python for Signal Processing“, Springer (2023).

33. R. K. Burdick, C. M. Borror, D. C. Montgomery: "Design and Analysis of Gauge R&R Studies", 1st Ed. SIAM (2005); S. H. Derakhshan , C. V. Deutsch: "Numerical Integration of Bivariate Gaussian Distribution", Paper 405, CCG Anual Report 13 (2011).

34. C. Paar, J. Pelzl: "Understanding Cryptography", Springer (2010); H. Delfs, H. Knebl: "Introduction to Cryptography", 3rd Ed. Springer (2015); J. Katz, Y. lindell: "Introduction to Modern Cryptography", 2nd Ed, CRC Press (2015); O. Goldreich: "Foundations of Cryptography", Cambridge University Press (2008); J. P. Aumasson: "Serious Cryptography", no starch press (2018).

35. Kaggle-link: competition-documentation: https://www.kaggle.com/competitions/drw-crypto-market-prediction. R. Nystrom: „Game Programming Patterns“, 1st Ed. genever benning (2014); A. A. Stepanov, D. E. Rose: „From Mathematics to Generic Programming“, 1st Ed. Addison-Wesley (2015);

36. J. Berk, P. DeMarzo: „Corporate Finance“, 6th Ed., Pearson (2023); R. W. Melicher, E. A. Norton: "Introduction to Finance", 16th Ed. WILEY (2017); Anatoly B. Schmidt: "Quantitative Finance for Physicists: An Introduction", 1st Ed. Academic Press (2005); Alex Backwell: "An Intuitive Introduction to Finance and Derivatives: Concepts, Terminology and Models", 1st Ed,

Springer (2023); Michael Isichenko: "Quantitative Portfolio Management: The Art and Science of Statistical Arbitrage", 1st Ed., Springer (2021); John H. Cochrane: "Asset Pricing", Revised Ed., Princeton University Press (2005); Antti Ilmanen: "Expected Returns: An Investor’s Guide to Harvesting Market Rewards", 1st Ed., WILEY (2011); Steven E. Shreve: "Stochastic Calculus for Finance I & II", 1st Ed., Springer (2004); Andrew Pole: "Statistical Arbitrage: Algorithmic Trading Insights and Techniques", 1st Ed., WILEY (2007); Mark S. Joshi: "The Concepts and Practice of Mathematical Finance", 2nd Ed., Cambridge University Press (2008);

37. E. Parzen: "Stochastic Processes", 3rd Ed. Dover Publications (2015); S. Aloorravi: "Metaprogramming with Python", 1st Ed. Packt (2022); B. Klein, P. Klein: "Funktionale Programmierung mit Python", Hanser (2025); K. Webel, D. Wied: "Stochastische Prozesse", 2. Auflage Springer (2016); L. Held: "Methoden der statistischen Inferenz", 1. Auflage Spektrum (2008); E. Cinlar: "Stochastic Processes", Dover (2013); N. Bäuerle, U. Rieder: "Finanzmathematik in diskreter Zeit", Springer-Spektrum (2017); M. Albrecht, R. Maurer: "Investment- und Risikomanagement", 3. Auflage, Schäffer Poeschel (2008); N. H. Bingham, R. Kiesel: "Risk Neutral Valuation: Pricing and Hedging of Financial Derivatives", 2. Auflage Springer (2004); T. Björk: "Arbitrage Theory in Continuous Time", 3rd Ed. Oxford University Press (2009); N. J. Cutland, A. Roux: "Derivative Pricing in Discrete Time", Springer (2013); F. Delbaen, W. Schachermayer: "The Mathematics of Arbitrage", Springer (2006); R. J. Elliott, P. E. Kopp: "Mathematics of Financial Markets", 2nd Ed. Springer (2005); H. Föllmer, A. Scheid: "A Stochastic Finance: An Introduction in Discrete Time", 3rd Ed. de Gruyter (2011); J. C. Hull: "Options, Futures and Other Derivatives", 8th Ed. Pearson (2011); J. Kremer: "Einführung in die diskrete Finanzmathematik", Springer (2005); D. Lamberton, B. Lapeyre: "Introduction to Stochastic Calculus Applied to Finance", Chapman & Hall (2007); D. G. Luenberger: "Investment Science", Oxford University Press (1998); S. R. Pliska: "Introduction to Mathematical Finance: Discrete Time Models", Blackwell (2000); A. N. Shiryaev: "Essentials of Stochastic Finance", World Scientific (2001); S. E. Shreve: "Stochastic Calculus for Finance I: The Binomial Asset Pricing Model", Springer (2004); J. Kremer: "Portfoliotheorie, Risikomanagement und die Bewertung von Derivaten", Springer (2011); L. Rüschendorf: "Mathematical Risk Analysis", Springer (2013).

38. A. Becker: "Kalman Filter - From the Ground Up", 1st Ed. private publication (2023); K. Triantafyllopoulos: "Bayesian Inference of State Space Models", 1st Ed. Springer (2021); P. Zarchan, H. Musoff: "Fundamentals of Kalman Filtering: A Practical Approach", 3rd Ed. AIAA (2009); A. Sidi: "Vector Extrapolation Methods with Applications", 1st Ed. SIAM (2019); C. Brezinski, M. R. Zaglia: "Extrapolation Methods - Theory and Practice", 2nd Ed. North-Holland (2002); C. Gardiner, P. Zoller: "Quantum Noise: A Handbook of Markovian and Non-Markovian Quantum Stochastic Methods with Applications to Quantum Optics", 3rd Ed. Springer (2004); K. Kendre: "Machine Learning for Quantum Noise Reduction", https://arxiv.org/abs/2509.16242 (2025); D. C. Marinescu, G. M. Marinescu: "Classical and Quantum Information", 1sr Ed. Academic Press (2012); Liao, H et al.: "Machine Learning for Practical Quantum Error Mitigation", arXiv:2309.17368v2 (2024), https://arxiv.org/pdf/2309.17368; Streamlit: https://streamlit.io/; Mitiq-package: https://quantum-journal.org/papers/q-2022-08-11-774/, https://arxiv.org/abs/2009.04417; Extrapolation packages: https://pypi.org/project/extrapolation/

39. A. Koop, H. Moock: "Lineare Optimierung - Eine anwendungsorientierte Einführung in Operations Research", 1st Ed. Spektrum (2008); G, B, Dantzig, M. N. Thalpa: "***Linear Programming 1: Introduction***", 1st Ed. Springer (1997) & "Linear Programming 2: Theory and Extensions", 1st Ed. Springer (2003); H. S. Kasana, K. D. Kumar: "Introductory Operations Research, Theory and Applications", 1st Ed. Springer (2004); D. G. Luenberger: "Linear and Nonlinear Programming", 2nd Ed. Kluwer (2004); R. J. Boucherie, A. Braaksma, H. Tijms: "Operations Research - Introduction to Models and Methods", 1st Ed. World Scientific (2022); A. J. King, S. W. Wallace: "Modeling with Stochastic Programming", 2nd Ed. Springer (2024); J. O. Royset, R. J.-B. Wets: "An Optimization Primer", 1st Ed. Springer (2021); cvxpy package: https://www.cvxpy.org/, https://pypi.org/project/cvxpy/; py-packages for operations research: https://wiki.python.org/moin/PythonForOperationsResearch